Linear Algebra for Machine Learning: A Quick and Easy Beginners Guide

1. What is Linear Algebra

Linear algebra as a whole is the study of linear functions and combinations, and extends further to the subject of abstract algebra, although in the context of linear algebra for machine learning you will mostly just see vectors and matrices. You can think of a vector as a finite list of numbers (or infinite, but you won't need to consider that case here), and a matrix as multiple vectors stacked on top of one another. In a programming context, you can also think of lists and/or arrays as vectors and matrices.

2. Why Learn Linear Algebra

Linear algebra is a crucial part of many subject areas, like engineering, physics, and of course machine learning. We use it to turn informational data into a data type that we can perform mathematical and statistical operations on, and then convert it back afterwards to get new and predictive information. For examples of where you will use linear algebra, check out the second last section of this article.

Some more complicated fields, like the aforementioned field of physics, as well as robotics, makes use of a combination of linear algebra and calculus, which is commonly seen in the subject of multivariable calculus. Robotics is also a natural next step for many curious machine learning enthusiasts, so if you wish to pursue that field then grasping the fundamentals of linear algebra for machine learning should be of high importance to you.

3. Vectors

Formally, a vector is a finite set of elements where each element belongs to a field, and the vector itself follows various different axioms that always hold true.

But luckily, when learning linear algebra for machine learning specifically, you don't need to know any of this mathematical jargon. All you really need to know is that a vector is a list of numbers, and that we can add and multiply vectors in a natural intuitive way.

Let's start with an example of a vector, which we will call v:

$$v = \langle 1, 2, 3 \rangle.$$

As you can see, a vector is typically written as a sequence of numbers inside angled brackets. Vectors can be any length, and will often be written with variables inside them instead of numbers, for example:

$$v = \langle x, y, z \rangle.$$

4. Matrices

Now let's introduce matrices, before getting into addition and multiplication. Since we are skipping mathematical formalities, you can think of a matrix as multiple vectors stacked on top of one another. We denote the dimension of a matrix as $m \times n$, where m is the number of rows (or vectors stacked on top of one another), and n is the number of columns (or the length of the vectors that are stacked). Note also that usually instead of angled brackets matrices will have either square or curved brackets. Consider the example below as a generalized matrix.

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}.$$

For generalized matrices, we need two indices to indicate the position of the elements, the first subscript indicating which row the element belongs to, and the other indicating the columns. Note the matrix need not have the same number of columns and rows. Here is an example of that:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}.$$

5. Addition

Vector Addition

Adding vectors occurs in a natural and intuitive way. Consider the vectors:

$$v_1 = \langle x_1, y_1, z_1 \rangle \text{ and } v_2 = \langle x_2, y_2, z_2 \rangle.$$

Then when we add v and w together we add the i^{th} components together to get another vector, where the i^{th} component refers to the index (or subscript in this case) of an element in the vector. So $v_1 + v_2 = \langle x_1 + x_2, y_1 + y_2, z_1 + z_2 \rangle$.

Let's do an example. Let's give values to the vectors above so that they become:

Then

$$v_1 + v_2 = \langle 1 + 4, 2 + 3, 3 + 2 \rangle = \langle 5, 5, 5 \rangle.$$

Matrix Addition

Matrix addition works the exact same way, so consider the following two matrices:

$$A = \begin{bmatrix} 5 & 7 \\ 1 & 2 \end{bmatrix} \text{ and } B = \begin{bmatrix} 8 & 9 \\ 3 & 4 \end{bmatrix}.$$

Then computing $A + B$ will give:

$$A + B = \begin{bmatrix} 5+8 & 7+9 \\ 1+3 & 2+4 \end{bmatrix} = \begin{bmatrix} 13 & 16 \\ 4 & 6 \end{bmatrix}.$$

And that's it for addition. Subtraction works in the same way except you replace all the additions with subtractions. As an exercise, can you work through $A - B$? Check your answer afterwards:

$$A - B = \begin{bmatrix} -3 & -2 \\ -2 & -2 \end{bmatrix}.$$

Note that the dimensions of both the vectors and matrices must be the same in order to add or subtract them.

6. Multiplication

Scalar Multiplication

Now onto multiplication. There are three types of multiplication, the first being scalar multiplication, which simply means multiplying a vector or matrix by a single value (i.e. scalar simply means singular value). For example, let your scalar variable be c, and let

$$v = \langle x, y, z \rangle \text{ and } A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}.$$

Then multiplying each by c gives:

$$c \cdot v = \langle c \cdot x, c \cdot y, c \cdot z \rangle \text{ and } c \cdot A = \begin{bmatrix} c \cdot a_{11} & c \cdot a_{12} \\ c \cdot a_{21} & c \cdot a_{22} \end{bmatrix}.$$

Indeed, multiplying both vectors and matrices by a scalar simply means to multiply each element inside them by that scalar. Now let us move on to the more complicated side of multiplication.

Dot Product

While not exactly a type of multiplication, the dot product is a very common operation performed on vectors that you must know, and will be needed in the next section. Luckily it is fairly simple, so consider two vectors again:

$$v_1 = \langle x_1, y_1, z_1 \rangle \text{ and } v_2 = \langle x_2, y_2, z_2 \rangle.$$

The dot product is then denoted by a thick dot, and will be computed by taking the products of the i^{th} components of each vector and summing them up, i.e.

$$v_1 \cdot v_2 = x_1 \cdot x_2 + y_1 \cdot y_2 + z_1 \cdot z_2.$$

A notable fact about the dot product is that when it is zero, it means that the vectors are perpendicular. In a physical sense, thinking about two vectors that represent directions in the physical plane, this means that the angle between two vectors is 90 degrees. There is much more detail and meaning behind the dot product, but for now this is all you will need.

Vector and Matrix Multiplication

Although this might seem a bit odd, let's start with matrix multiplication first. If you are multiplying two matrices, say A and B with size $m_1 \times m_2$ and $n_1 \times n_2$ respectively, then you must have that $m_2 = n_1$ or else you cannot perform the multiplication. The resulting matrix will also be of dimension $m_1 \times n_2$. This follows naturally by how it works, so let's do an example by considering the following matrices:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \text{ and } B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}.$$

Now think about each column or row of a matrix as a vector. Then in the resulting matrix the element in the i^{th} row and j^{th} column will be the dot product of the i^{th} row vector in matrix A and the j^{th} column vector in matrix B. So the result will look like:

$$A = \begin{bmatrix} a_{11} \cdot b_{11} + a_{12} \cdot b_{21} & a_{11} \cdot b_{12} + a_{12} \cdot b_{22} \\ a_{21} \cdot b_{11} + a_{22} \cdot b_{21} & a_{21} \cdot b_{11} + a_{22} \cdot b_{22} \end{bmatrix}.$$

While this may look complicated, remember each entry is just a scalar value, so if you had values to fill in for the variables you will end up with a more simple looking matrix like before. Also note for higher dimensional matrices the same formula is applied, but you will just have more tedious calculations to do, with a larger resulting matrix.

Now that you know matrix multiplication, seeing how vector multiplication is easy, because a vector is in fact just a matrix with dimension $1 \times n$. This means that a vector is just a matrix with one row and n columns. You can also look at a vector as a column vector with dimension $n \times 1$, so it will have n rows this time with only one column. Now multiplying a vector by a matrix is just like before, but here is an example anyways. Let $v = \langle x, y \rangle$. Then

$$v \cdot A = \begin{bmatrix} x \cdot a_{11} + y \cdot a_{21} & x \cdot a_{12} + y \cdot a_{22} \end{bmatrix}.$$

As you can see this is a matrix of size 1×2, which can also be seen as a vector. Thus, you can use square or angled brackets here. It doesn't really matter and in the common literature you may read the type of brackets for vector and matrix notation will often be used interchangeably.

The final thing to mention is something notable which is when you multiply a vector of dimension $n \times 1$ by a vector of dimension $1 \times m$, in which case the result of this operation will be a matrix of dimension $n \times m$. Consider the same v above for example, with a column vector or matrix

$$w = \begin{bmatrix} a \\ b \\ c \end{bmatrix}.$$

Then

$$w \cdot v = \begin{bmatrix} x \cdot a & y \cdot a \\ x \cdot b & y \cdot b \\ x \cdot c & y \cdot c \end{bmatrix}.$$

This operation in certain cases is called the outer product, and serves as the opposite of the dot product.

Cross Product

Although the cross product may not show up too often in the context of linear algebra for machine learning, it is a very common and important operation to know, and will surely come up at some point during your career. The cross product is just an operation on two 3 dimensional vectors, or equivalently vectors of length 3.

Consider the vectors from before,

$$v_1 = \langle x_1, y_1, z_1 \rangle \text{ and } v_2 = \langle x_2, y_2, z_2 \rangle,$$

then the cross product is denoted as

$$v_1 \times v_2 = \langle x_3, y_3, z_3 \rangle.$$

To get the values of x_3, y_3, and, z_3, simply follow the following formula.

$$\begin{aligned} x_3 &= y_1 z_2 - z_1 y_2 \\ y_3 &= z_1 x_2 - x_1 z_2 \\ z_3 &= x_1 y_2 - y_1 x_2 \end{aligned}$$

Since the cross product is an operation only in the 3-d plane, you can imagine it has real world implications, namely that it represents a vector that is perpendicular to both the vectors v_1 and v_2. A couple facts about it that may come in handy: first is that if v_1 and v_2 are parallel to each other, then their cross product will be a vector of all zeroes. The second is that it is anti-commutative, meaning that $v_1 \times v_2 = -v_2 \times v_1$. There is tons of more free information on the cross product that you can look up if need be, but in regards to linear algebra for machine learning, this should suffice for now.

7. Determinant

Another operation that doesn't really resemble multiplication or any other typical operation is the determinant. While this may be a concept you won't use too much in the context of linear algebra for machine learning, it is still a fundamental concept and also one that is necessary to understand invertible matrices (introduced in the next section).

You can think of the determinant as a function that returns a scalar value, given an input that is a square matrix. To calculate the determinant, we first need to look at it for a 2×2 matrix:

$$M = \begin{bmatrix} a & b \\ c & d \end{bmatrix}.$$

The determinant is then calculated to be simply $\det M = ad - bc$, and is commonly denoted by both $\det M$ and $|M|$.

To see how this translates to larger matrices, let us extend this to a 3×3 matrix:

$$M = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}.$$

Basically to compute $|M|$ we must break it down into smaller chunks, which is done by picking a row or column in the matrix, and then for each element of your chosen row or column, multiply it by the determinant of a 2×2 matrix that is made up of elements that do not share the same row or columns as that element. Also, for the elements in your chosen row or column that had an even indexed position, multiply them by -1, and then sum all these products up to get your determinant.

Surely reading that for the first time sounds complicated, but it really isn't once you see how it is done visually, so let's calculate the determinant of M. Note that when computing the determinant of a matrix, we often swap the square or curly brackets with the vertical lines or pipes as in the $|M|$ notation:

$$\begin{aligned} |M| &= \begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} \\ &= a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix} \\ &= a(ei - fh) - b(di - fg) + c(dh - eg) \\ &= aei + bfg + cdh - (afh + bdi + ceg). \end{aligned}$$

This process can naturally extend to larger matrices in a similar fashion, by continually breaking down the matrix in a recursive pattern. That being said, this level of understanding should suffice for now, and one of the most important facts regarding the determinant is its relation to other types of matrices, like invertible matrices, which we will move onto now.

8. Other Types of Matrices

There are other notable types of matrices that are important in the context of linear algebra for machine learning that can be obtained by performing some kind of operation or transformation to any matrix. The most important ones are listed here, and once you have a grasp for the basics of this linear algebra material, feel free to check out some of the other types of matrices.

Transpose

The first is the transpose of a matrix. To make the next concepts more clear, consider a 3×3 dimensional matrix now:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}.$$

Then the transpose of a matrix is obtained by swapping the off diagonal elements with each other. More formally, this means that we swap the ij^{th} elements with the ji^{th} elements. Take a look at $A's$ transpose to see this more clearly, where we denote the transpose with a superscript T.

$$A^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \\ a_{13} & a_{23} & a_{33} \end{bmatrix}.$$

This can also work for matrices that have dimension $m \times n$ where m and n are not equal to each other. In this case the resulting matrix will have dimension $n \times m$. Consider the previous matrix A but with the third column removed, i.e.

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}.$$

Then

$$A^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \end{bmatrix}.$$

Furthermore, this extends to vectors as well since they are often seen as a one column or one row matrix. In fact, in a lot of math textbooks the standard way to write a vector will be as a row, like $v = \langle x, y, z \rangle$, and then when the author wants you to think of the vector as a column vector they will write it as its transpose:

$$v^T = \begin{bmatrix} x \\ y \\ z \end{bmatrix}.$$

Identity Matrix

Just a quick definition here, the identity matrix is something you must know and basically acts as the 1 value in matrix multiplication, meaning that multiplying any matrix or vector by the identity matrix will result in the same matrix or vector you started with. It is defined as follows:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

This of course scales to any dimension you wish, but note that it will always have the same number of rows and columns (a square matrix), and the diagonal elements will always be 1 and everything else will be 0.

Invertible Matrix

Think about a number, say 5, then the inverse of 5 would be 1/5, and multiplying 5 by its inverse will give you 1. Well an invertible matrix is simply a matrix that has a similar inverse matrix like the 1 / 5, such that by multiplying the matrix by its inverse you will get the identity matrix.

Moreover, a matrix is categorically called an invertible matrix if and only if it indeed has such an inverse. The reason for stating this again is because it is not guaranteed that a matrix will have an inverse, and thus may not be an invertible matrix. So how do we know if a matrix is invertible? Well luckily you need only check two things. Firstly, the matrix must be a square, and secondly, the determinant of the matrix must be zero, which you already know how to do now. There is so much more to learn about invertible matrices, and they have many more uses in other fields. While it may not be necessary, I would recommend at some point reading up on invertible matrices a bit more.

9. Examples of Linear Algebra for Machine Learning

Now that we've covered all of the fundamentals of linear algebra for machine learning, let's move on to the fun part, which is where and when we actually get to use this stuff. Well, below are just a few common examples, but do keep in mind learning about these applications in detail will be quite a challenge if you have not yet fully grasped the above material.

One-Hot Encoding

Sometimes in machine learning you will have data that may be troublesome to handle. One common way to deal with this is with one-hot encoding. Say you have multiple classes of objects, then you would label each class with a vector filled with all zeros except for one element, which will be a 1 in a unique spot/index in the vector relative to all other classes (note: this means that the dot product between any two of the one-hot encoded vectors will be zero).

Principal Component Analysis

A common concept in machine learning that you will learn early on is principal component analysis, or PCA for short. As its name suggests, the aim of PCA is to find variables that give the most amount of information, and then discard other variables that are more or less redundant. This way, you reduce the dimensionality of your problem, which essentially means it simplifies the problem.

The linear algebra used here can get a bit more complicated than what was given in this post, but the fundamentals explained above should enable you to understand any extra necessary material quickly. (Quick tip: read up on eigenvalue and eigenvectors before looking into PCA).

Neural Networks

Perhaps the most popular occurrence of the linear algebra for machine learning you learned here is in deep learning and neural networks. If you haven't yet introduced yourself to neural networks, I encourage you to do so soon, as it is truly the hottest topic in all of machine learning and artificial intelligence. But once you do you will see that it is a collection of nodes and lines that connect them, and these lines have weights attached to them which will be stored inside a matrix. Computations performed when training a neural network will largely be matrix addition and multiplication, so honing in on your skills in linear algebra for machine learning in particular will come in handy.

Computer Vision

Computer vision is a branch of machine learning that typically deals with unstructured data in the form of images. In fact, an image is simply a matrix filled with values that describe the color of each pixel in the image, so naturally you can already see the material you learned in this linear algebra for machine learning article coming into play here. You then create predictive models using convolutional neural networks, which are a special type of neural network typically reserved for images.

In computer vision the output of your algorithm may be another image, like an altered version of your input image, or a word or number describing the original image, like a description of the type of animal in your input image. What you get out of you convolution neural networks really depends on the type of problem you have, but there are a wide variety of problems being solved with these methods, so definitely look into later if this sounds interesting.

Natural Language Processing

Natural language processing (NLP) is another branch of machine learning and typically deals with words as your input data. But how do we convert words into data that we can train predictive models on? Well you can use linear algebra of course, by changing each word into a vector using something called word embeddings. Similar to computer vision, which has its own special type of neural network, NLP problems tend to use its own special type of neural network, called recurrent neural networks. The details on how these work go pretty in-depth, so I'll skip it here, but either way you can clearly see now how important it is to learn the necessary linear algebra for machine learning as it has so many uses in this field, so it's never too early to start learning it!